Orbi

Live device and connection popup for Windows 11: product and technical plan

Version 0.1 · 2 Nov 2026

1. Vision

Orbi is a tiny always-on Windows app. When something connects (earbuds, Wi-Fi, Ethernet, a USB device), a polished popup slides up showing a live visual of it, like the AirPods popup on a MacBook: which part is charged, which is not. It must look like the real thing, use almost no RAM or CPU, and work for any connection type, not just AirPods.

2. Goals and constraints

• Very low RAM and about 0% idle CPU. No polling; react to Windows events only.

• Looks and feels like the real Apple popup (animations, blur, rounded card, device artwork).

• Generic: Bluetooth, Wi-Fi, Ethernet, USB and more, through a plugin model.

• Native to Windows 11: tray app, auto-start, system theme, notifications.

• Readable, maintainable code with a clear plugin boundary.

3. Features

3.1 Core (v1)

• Auto popup on connect, auto-dismiss after about 5 s, hover to keep open.

• Bluetooth earbuds: battery level with animated ring; low-battery warning color.

• AirPods: left bud, right bud and case battery, charging state, with per-part visual (from BLE advertisements).

• Wi-Fi: SSID, signal bars, band and speed on connect.

• Ethernet: link speed, connected or disconnected state.

• USB: device name, type icon, drive space for storage devices.

• Tray icon with tooltip summary and context menu (show now, settings, quit).

• Start with Windows; runs hidden.

3.2 Later (v2+)

• Live popup that updates while open (battery change, signal change).

• Per-device custom artwork packs and user-supplied mockups.

• Settings window: enable or disable device types, popup position, duration, theme.

• Notifications for low battery and disconnects; history log.

• Other transports: Bluetooth LE peripherals, hotspot, VPN, monitors, controllers.

• Optional Capacitor or mobile companion reusing the same UI.

4. Tech stack decision

Option	Idle RAM	Popup RAM	Windows fit	Verdict
Electron	150 MB+	150 MB+	Good	Rejected: heavy, always loaded
Tauri 2 (popup on demand)	5-15 MB	about 60 MB for 5 s	Very good, uses WebView2	**Chosen**
WinUI 3 / C#	30-60 MB	30-60 MB	Best	Fallback if Tauri popup RAM is a problem
Pure Rust + Direct2D	5-10 MB	10-20 MB	Best	Most work; hard to match polished visuals
Capacitor	n/a	n/a	Mobile only	Not for Windows tray apps

4.1 Chosen stack

• **Shell:** Tauri 2 (Rust). The main process is only a hidden event loop plus tray.

• **Popup UI:** HTML, CSS and SVG, with a small framework-free TypeScript layer (or Svelte if state grows). WebView2 window created per event and destroyed after dismiss.

• **Windows APIs:** the `windows` Rust crate (WinRT and Win32).

• **Packaging:** Tauri bundler to MSI or NSIS installer; autostart plugin.

• **Tooling:** Rust stable, Node 24, npm, Vite for the UI.

5. Architecture

Three layers. **Watchers** (Rust) subscribe to Windows events and emit a normalized `DeviceEvent`. The **core** deduplicates, decides whether to show a popup and tracks live state. The **popup** (WebView) renders by device type.

Layer	Responsibility	Details
Watchers	Detect connect, disconnect, change	One module per source implementing a common trait; push-only, no timers
Core	State and routing	In-memory map of live devices; debounce; popup rules; tray tooltip
Popup	Render and animate	Receives JSON over Tauri events; layout chosen by `type`
Settings	User options	JSON file in AppData; loaded at start

5.1 Normalized device event

```
{ id, type: bluetooth|wifi|ethernet|usb, name, icon, status:
connected|disconnected|updated, parts: [{ key: left|right|case|main, level:
0-100, charging: bool }], metrics: { ssid, signal, speed, free_space } }
```

5.2 Event sources (zero polling)

Source	Windows API	Gives
Bluetooth connect	WinRT `DeviceWatcher` / `BluetoothLEAdvertisementWatcher`	Connect, disconnect, name
Bluetooth battery	GATT Battery Service notifications; device property key 104EA319-...-2 for classic	Battery percent
AirPods parts	BLE advertisement manufacturer data (Apple company ID 0x004C, proximity pairing message)	Left, right, case level and charging
Wi-Fi	`WlanRegisterNotification` (WLAN API)	SSID, signal quality, connect events
Ethernet / IP	`NotifyIpInterfaceChange`, `GetIfEntry2`	Link up/down, speed
USB / hardware	`CM_Register_Notification`	Arrival, removal, class, name

6. UI and design

• Card about 340 x 220, bottom center above the taskbar, rounded 28 px, dark translucent background with blur (Mica or acrylic where available).

• Slide-up spring animation in, fade out; respects Windows reduced motion.

• Device hero artwork (SVG): earbuds with each bud and the case filled according to charge, ring or bar per part.

• Color rules: green above 50 percent, yellow 21-50, red 20 and below; charging shows a bolt.

• Wi-Fi shows animated signal bars; Ethernet shows a plug and speed; USB shows a type icon and storage bar.

• Follows system light or dark theme; Segoe UI Variable typography.

• Does not steal focus (shown inactive) and never blocks input.

• Artwork is swappable: your custom mockups and AirPods reference images go in an assets folder keyed by device model.

7. Performance budget

Metric	Target
Idle RAM (tray process)	under 15 MB
Idle CPU	about 0% (event driven)
Popup RAM (visible)	under 80 MB, released after dismiss
Popup latency	under 300 ms from event to visible
Installer size	under 10 MB

8. Project structure (as built)

```
orbi/
  src-tauri/src/ lib.rs (tray, wiring), events.rs (DeviceEvent), core.rs (state, rules, tests),
    pipeline.rs (watchers to core to popup), popup.rs (on-demand window), mock.rs
  src-tauri/src/watchers/ mod.rs (Watcher trait); bluetooth, airpods, wifi, ethernet, usb come in chunks 4-6
  src/ main.ts (render by device type), styles.css; index.html at the root
```

```
prototype-electron/ old throwaway prototype
Orbi-Tech-Plan.pdf this document
```

9. Roadmap

Phase	Scope	Outcome
0	Install Rust, scaffold Tauri, hidden tray app, popup on demand with mock data	Popup looks right, idle RAM measured
1	Bluetooth watcher and generic battery	Real earbuds trigger the popup
2	AirPods BLE parsing: left, right, case	Per-part visual works
3	Wi-Fi, Ethernet, USB watchers	Generic across connection types
4	Polish: animations, themes, your mockups, settings, autostart	Matches the real popup
5	Installer, testing across devices, optimization	Shippable v1

10. Risks and notes

• Windows exposes battery for many but not all Bluetooth devices; case and bud levels exist only for AirPods-style advertisements (Apple format is undocumented and reverse-engineered).

• AirPods artwork and Apple trademarks: use your own artwork or generic earbuds in the shipped product.

• WebView2 popup memory is the main cost; if too high, switch popup to a native renderer.

• BLE advertisement scanning needs a radio that supports it; test on your hardware.

• Bluetooth may prompt for permission on first use.

11. Open items

• Your mockups plus the AirPods reference images.

• Confirm Rust toolchain installation.

• Which earbuds you own, for real-device testing.

12. Decisions log (what we agreed)

• Product: Orbi, a Windows version of the MacBook AirPods popup showing a visual of the device and which part is charged.

• Platform: Windows 11 desktop (chosen over web page and macOS).

• Devices: any Bluetooth earbuds, plus Wi-Fi, Ethernet and USB. Not limited to AirPods.

• Options your senior mentioned: Windows native, Tauri, Capacitor. Decision: Tauri. Capacitor is mobile-only; Windows native is the fallback.

• Priorities: minimal RAM and CPU, great Windows integration, readable, automatic, live updates, look as close to the real popup as possible.

• Design input: you will supply AirPods reference mockups from the internet plus your own mockups.

• Status: chunks 1-3 are built and verified (see section 14). The Electron prototype is parked in prototype-electron/ and will be deleted. Chunks 4-8 are next.

13. Build chunks (approved plan)

Chunk	Scope	Verify
1	Rust toolchain check, Tauri 2 + Vite scaffold, move Electron prototype to prototype-electron/, hidden tray app with Show now and Quit	Tray icon visible, idle process
2	Popup on demand (frameless, transparent, non-focusable, destroyed after about 5 s), port card UI, DeviceEvent type, mock events per type	Popup shows and hides; RAM measured against budget
3	Core (live-device map, dedupe, debounce, tooltip) and Watcher trait	cargo test
4	Bluetooth watcher: DeviceWatcher plus battery property and GATT	Real earbuds connect and show battery
5	AirPods BLE parsing: left, right, case, charging; per-part artwork	Parser tests; real AirPods if owned
6	Wi-Fi, Ethernet, USB watchers and layouts	Toggle Wi-Fi, unplug cable, plug USB
7	Polish: your mockups, Mica, animation, themes, settings, autostart	Visual match to reference
8	Installer, final RAM and CPU measurement	Meets performance budget

I stop for your review after each chunk.

14. Progress log

Chunk	Status	What was built and verified
1	Done	Rust 1.99.1 and VS C++ Build Tools installed. Tauri 2 scaffold, tray-only app (no window at start), tray menu. Needed Smart App Control turned off, since it blocked unsigned Rust build helpers.
2	Done	Popup window created on demand and destroyed after about 5 s (hover keeps it open). Frameless, transparent, always on top, no taskbar entry, never takes focus, placed bottom center above the taskbar using the monitor work area and DPI scale. UI layouts for earbuds, AirPods (left, right, case), Wi-Fi, Ethernet, USB. Verified by screenshots on a 1920x1080 display at 150 percent scale.
3	Done	events.rs (normalized DeviceEvent), core.rs (live-device map, 2 s reconnect debounce, popup only for new devices and for a part newly dropping to 20 percent or below, tray tooltip), Watcher trait, pipeline thread. 7 unit tests pass (cargo test). End to end run: popup showed about 3 s after launch and closed on schedule.
4	Built, hardware unverified	Bluetooth watcher: WinRT DeviceWatcher wakes a worker; battery read from the Windows device property; re-read every 60 s only while a device is connected. Compiles and runs; this PC had no battery-reporting earbuds.
5	Built, hardware unverified	AirPods parser for Apple BLE proximity-pairing data (left, right, case, charging) with RSSI filter. 4 unit tests on hand-built payloads. The BLE scan reported device not ready on this PC (Bluetooth radio likely off). Left and right order must be confirmed on real AirPods.
6	Done	Wi-Fi and Ethernet via NotifyIpInterfaceChange (virtual adapters filtered), USB removable drives via CM_Register_Notification on volumes. Live Wi-Fi read verified on this PC (SSID, signal, speed). Event firing not yet exercised with a real plug or unplug.
Design	Done (v1)	Orbit design language: every device is an object inside a ring whose arc is the key value, with a glowing dot riding the arc end. Verified by screenshots for AirPods, single earbud, Wi-Fi, Ethernet, USB.
7-8	Not started	Mica blur and your mockups, settings, autostart, installer, release-build measurements.

14.1 Measurements (debug build, not optimized)

Metric	Measured	Budget
Tray process, idle	16-25 MB	under 15 MB (release build should be lower; to be re-measured in chunk 8)
Process with popup open	about 31 MB total (WebView2 child processes not included)	under 80 MB popup
CPU idle	about 0 (0.03-0.14 s total over 6-7 s, mostly startup)	about 0%

Metric	Measured	Budget
Popup latency	visible in under 3 s of launch in tests; not yet precisely timed	under 300 ms (to be measured)

14.2 Known issues and notes

• Memory figures cover only orbi.exe. WebView2 runs in separate msedgewebview2.exe processes while the popup is open; their total has not been measured yet.

• The popup card is a plain translucent dark card. Mica or acrylic blur is planned for chunk 7.

• The Rust compiler warns that Disconnected and Updated are never constructed; real watchers in chunk 4 will use them.

• Dev aids: run orbi.exe --mock=mock_airpods (also mock_earbuds, mock_wifi, mock_ethernet, mock_usb) or use the tray Mock menu.

• Build for testing with: npm run tauri build -- --debug --no-bundle. A plain cargo build makes the popup load the dev server and show a blank error page.

• Devices already connected when Orbi starts do not pop up; only new connections do.

• USB hard drives and SSDs are not detected yet because Windows reports them as fixed disks.

• Chunks 4 and 5 need real earbuds and AirPods before they can be called verified.

• Repo: https://github.com/goprocker/Orbi (public).

14.3 Design: orbit language

• Inspiration reviewed: the macOS AirPods popup (dark card, product render, battery pill per part), AirBuddy style ring gauges and state colors, and Windows clones that copy the Apple pill.

• Orbi's own idea: the name is the design. Each device is an object inside a thin orbit ring; the arc length is the main value (battery, signal, disk used) and a glowing dot rides the arc end.

• Palette: ink 14161B, graphite 1E2128, porcelain ECEEF2, ion 5BE3A8 (healthy), amber F5B94A (middle), flare FF6B5E (low).

• Type: Segoe UI Variable (native, no font cost). Light-weight large numerals, semibold names, sentence case.

• Motion: one orchestrated moment. The card rises, the arcs sweep in with the dots, and the numbers count up together. Reduced motion is respected.

• Layouts: parts devices (left bud, right bud, case) show a title plus three orbits; everything else shows one large orbit with name, value and status.

• Controls: a close button appears on hover; hovering keeps the popup open.